

```
In [1]: import numpy as np
import pandas as pd
import random
```

```
In [2]: roll_no=np.arange(1,11)
m=np.random.randint(50,100,10)
a=np.random.randint(30,100,10)
g=np.array(['M','F','M',None,'M','F','M','F','M',None])
df=pd.DataFrame({
    'Roll_No':roll_no,
    'Marks':m,
    'Attendance':a,
    'gender':g
})
print(df)
```

	Roll_No	Marks	Attendance	gender
0	1	90	57	M
1	2	80	96	F
2	3	53	84	M
3	4	69	79	None
4	5	77	39	M
5	6	52	86	F
6	7	95	95	M
7	8	77	78	F
8	9	84	51	M
9	10	61	60	None

```
In [3]: df['gender'].isnull().sum()
```

```
Out[3]: np.int64(2)
```

```
In [5]: df['gender'].fillna(df['gender'].mode()[0],inplace=True)
```

/var/folders/s6/z415bvz54yvfxmhwk6xdk12c0000gn/T/ipykernel_1384/2533697725.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['gender'].fillna(df['gender'].mode()[0],inplace=True)
```

```
In [6]: df
```

```
Out[6]:
```

	Roll_No	Marks	Attendance	gender
0	1	90	57	M
1	2	80	96	F
2	3	53	84	M
3	4	69	79	M
4	5	77	39	M
5	6	52	86	F
6	7	95	95	M
7	8	77	78	F
8	9	84	51	M
9	10	61	60	M

```
In [7]: df['Marks'].mean()
```

```
Out[7]: np.float64(73.8)
```

```
In [10]: df['gender'].value_counts()
```

```
Out[10]: gender
M      7
F      3
Name: count, dtype: int64
```

```
In [16]: import matplotlib.pyplot as plt
gender_count=df['gender'].value_counts()
plt.figure()#creates a blank canvas (empty page) where your graph will be drawn.
plt.bar(gender_count.index,gender_count.values)
# plt.bar() → creates a bar graph

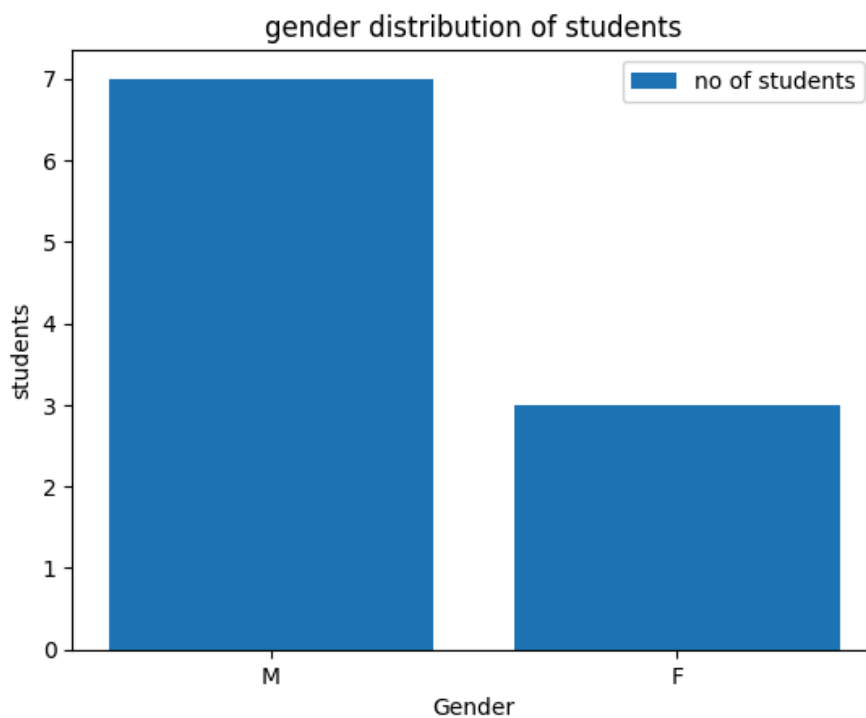
#gender_count.index → gives the categories (Male, Female) → X-axis

#gender_count.values → gives the counts (6, 4) → Y-axis
'''
plt.bar(x_values, y_values)
Where:

x_values → categories (X-axis)

y_values → numbers (Y-axis)
'''

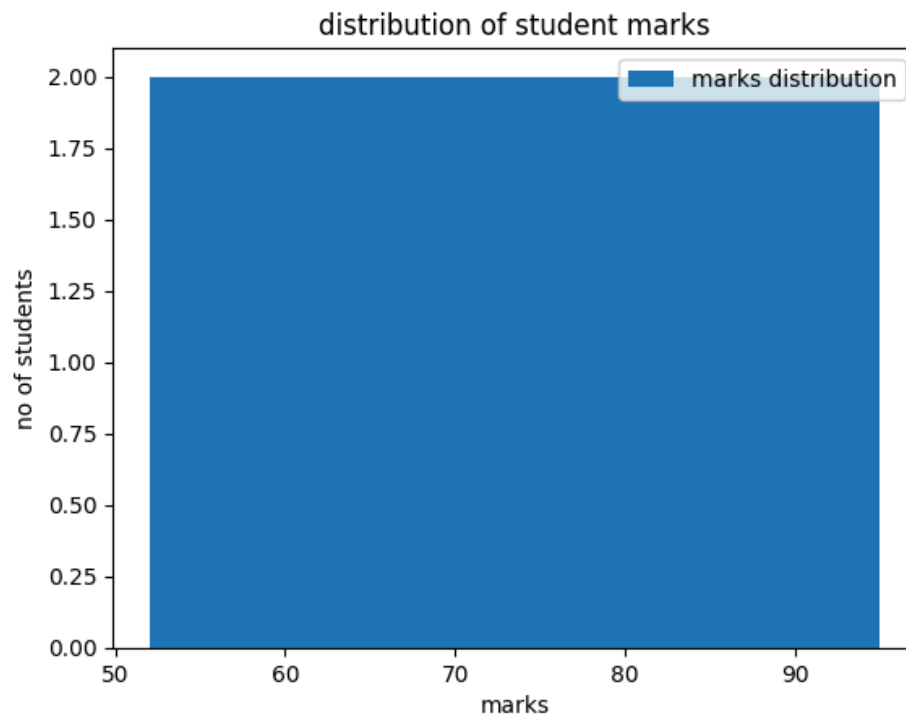
plt.xlabel("Gender")
plt.ylabel("students")
plt.title("gender distribution of students")
plt.legend(["no of students"])#Adds a legend box explaining what the bars represent.
plt.show()
```



```
In [18]: plt.figure()
plt.hist(df['Marks'],bins=5)#plt.hist() → Function used to draw a histogram.

#students['Marks'] → Selects the Marks column from the dataset.

bins=5 → Divides the marks into 5 groups (intervals).
plt.xlabel("marks")
plt.ylabel("no of students")
plt.title("distribution of student marks")
plt.legend(["marks distribution"])
plt.show()
```



```
In [20]: import random
l=[]
for i in range(13):
    m=np.random.randint(0,100)
    l.append(m)
print(l)
```

```
[29, 5, 88, 18, 58, 38, 34, 35, 82, 78, 5, 66, 51]
```

```
In [22]: m=np.random.randint(0,100,13)
m
```

```
Out[22]: array([28, 11, 55, 76, 20, 51, 75,  3, 67, 98, 69, 11, 78])
```

```
In [3]: #dfs
def dfs(start,goal,graph,vis=None):
    if vis is None:
        vis=set()
    print(start)
    if start==goal:
        return True
    vis.add(start)
    for child in graph[start]:
        if child not in vis:
            if dfs(child,goal,graph,vis):
                return True
    return False
```

```
In [9]: graph={
    'A':['B','C'],
    'B':['D','E'],
    'C':['F','G'],
    'D':[],
    'E':[],
    'F':[],
    'G':[]
}
dfs('A','D',graph)
```

```
A
B
D
```

```
Out[9]: True
```

```
In [10]: graph={
    'A':['B','C'],
```

```

    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}
dfs('A', 'E', graph)

```

A
B
D

Out[10]: False

```

In [4]: def dfs(start, goal, graph, vis=None):
        if vis is None:
            vis=set()
        print(start)
        if start==goal:
            return True
        vis.add(start)
        for child in graph[start]:
            if child not in vis:
                if dfs(child, goal, graph, vis):
                    return True
        return False

graph={
    'A': ['C', 'B'],
    'B': ['J', 'K'],
    'C': ['D', 'E'],
    'D': ['F', 'G'],
    'E': ['H', 'I'],
    'F': [],
    'G': [],
    'H': [],
    'I': []
}
dfs('A', 'F', graph)

```

A
C
D
F

Out[4]: True

```

In [5]: graph={
    'A': ['C', 'B'],
    'B': ['J', 'K'],
    'C': ['D', 'E'],
    'D': ['F', 'G'],
    'E': ['H', 'I'],
    'F': [],
    'G': [],
    'H': [],
    'I': []
}
dfs('A', 'G', graph)

```

A
C
D
F

Out[5]: False

```

In [12]: def bfs(st, goal, graph):
        vis=set()
        queue=[st]
        path=[]
        vis.add(st)
        while queue:
            node=queue.pop(0)
            path.append(node)

```

```

    if node==goal:
        return path
    for neighbour in graph.get(node,[]):
        if neighbour not in vis:
            vis.add(neighbour)
            queue.append(neighbour)
    return path

```

```

In [13]: graph={
    'A':['B','C','D'],
    'B':['E','F'],
    'C':['G','H'],
    'D':['I'],
    'E':['J','K'],
    'K':['N'],
    'N':[],
    'G':['L'],
    'H':[],
    'I':['M'],
    'M':[]
}
bfs('A','M',graph)

```

```

Out[13]: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M']

```

```

In [14]: import numpy as np
import pandas as pd
import random

```

```

In [18]: roll_no=np.arange(1,51)
name=["aadya","aakash","krishna","khushi","masoom","kiran","akshara","sugeeta","shalini","aash
weight=np.random.randint(50,100,50)
height=np.random.randint(100,150,50)
stipend=np.random.randint(500,1000,50)
df=pd.DataFrame({
    'R':roll_no,
    'N':name,
    'H':height,
    'S':stipend
})
print(df)

```

	R	N	H	S
0	1	aadya	119	890
1	2	aakash	131	577
2	3	krishna	109	678
3	4	khushi	129	525
4	5	masoom	129	684
5	6	kiran	127	607
6	7	akshara	101	700
7	8	sugeeta	129	687
8	9	shalini	135	738
9	10	aashna	118	525
10	11	aadya	108	793
11	12	aakash	139	517
12	13	krishna	126	986
13	14	khushi	111	920
14	15	masoom	117	729
15	16	kiran	101	960
16	17	akshara	128	898
17	18	sugeeta	108	552
18	19	shalini	143	702
19	20	aashna	132	662
20	21	aadya	102	644
21	22	aakash	115	688
22	23	krishna	115	900
23	24	khushi	103	767
24	25	masoom	132	948
25	26	kiran	148	951
26	27	akshara	123	750
27	28	sugeeta	117	674
28	29	shalini	141	831
29	30	aashna	105	839
30	31	aadya	120	526
31	32	aakash	127	671
32	33	krishna	147	599
33	34	khushi	128	787
34	35	masoom	134	689
35	36	kiran	103	531
36	37	akshara	110	676
37	38	sugeeta	101	644
38	39	shalini	133	945
39	40	aashna	124	600
40	41	aadya	126	657
41	42	aakash	139	795
42	43	krishna	120	923
43	44	khushi	126	957
44	45	masoom	125	572
45	46	kiran	101	561
46	47	akshara	133	697
47	48	sugeeta	122	546
48	49	shalini	103	696
49	50	aashna	101	788

```
In [23]: cols=['H','S']
for col in cols:
    df[col]=(df[col]-df[col].min()/(df[col].max()-df[col].min()))
print(df)
```

	R	N	H	S
0	1	aadya	0.382979	0.795309
1	2	aakash	0.638298	0.127932
2	3	krishna	0.170213	0.343284
3	4	khushi	0.595745	0.017058
4	5	masoom	0.595745	0.356077
5	6	kiran	0.553191	0.191898
6	7	akshara	0.000000	0.390192
7	8	sugeeta	0.595745	0.362473
8	9	shalini	0.723404	0.471215
9	10	aashna	0.361702	0.017058
10	11	aadya	0.148936	0.588486
11	12	aakash	0.808511	0.000000
12	13	krishna	0.531915	1.000000
13	14	khushi	0.212766	0.859275
14	15	masoom	0.340426	0.452026
15	16	kiran	0.000000	0.944563
16	17	akshara	0.574468	0.812367
17	18	sugeeta	0.148936	0.074627
18	19	shalini	0.893617	0.394456
19	20	aashna	0.659574	0.309168
20	21	aadya	0.021277	0.270789
21	22	aakash	0.297872	0.364606
22	23	krishna	0.297872	0.816631
23	24	khushi	0.042553	0.533049
24	25	masoom	0.659574	0.918977
25	26	kiran	1.000000	0.925373
26	27	akshara	0.468085	0.496802
27	28	sugeeta	0.340426	0.334755
28	29	shalini	0.851064	0.669510
29	30	aashna	0.085106	0.686567
30	31	aadya	0.404255	0.019190
31	32	aakash	0.553191	0.328358
32	33	krishna	0.978723	0.174840
33	34	khushi	0.574468	0.575693
34	35	masoom	0.702128	0.366738
35	36	kiran	0.042553	0.029851
36	37	akshara	0.191489	0.339019
37	38	sugeeta	0.000000	0.270789
38	39	shalini	0.680851	0.912580
39	40	aashna	0.489362	0.176972
40	41	aadya	0.531915	0.298507
41	42	aakash	0.808511	0.592751
42	43	krishna	0.404255	0.865672
43	44	khushi	0.531915	0.938166
44	45	masoom	0.510638	0.117271
45	46	kiran	0.000000	0.093817
46	47	akshara	0.680851	0.383795
47	48	sugeeta	0.446809	0.061834
48	49	shalini	0.042553	0.381663
49	50	aashna	0.000000	0.577825

In []: